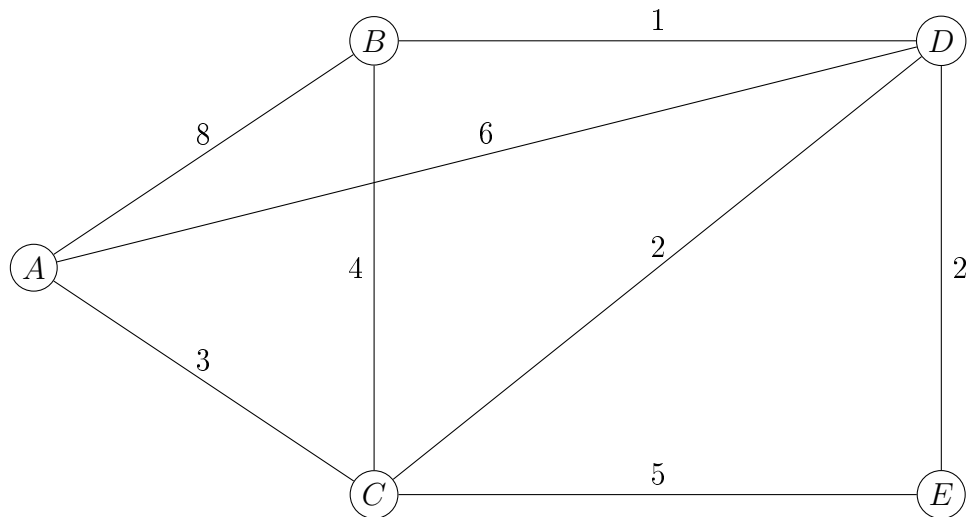


- TD 13 -

Algorithme de Dijkstra

Exercice 1 : Pour s'entraîner au tableau des poids minimaux

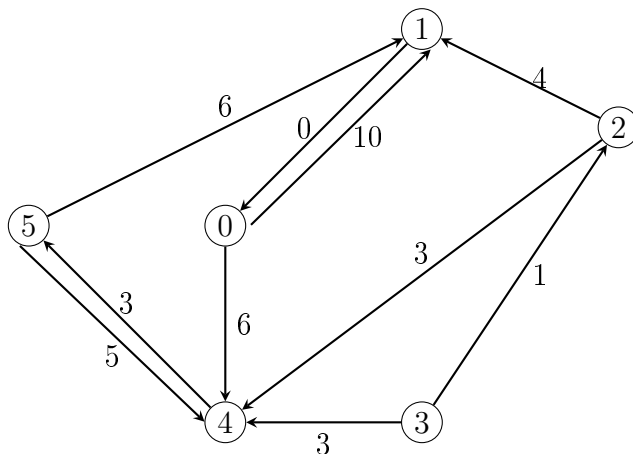
1. On considère le graphe ci-dessous :



Compléter le tableau d'exploration des sommets obtenu à l'aide de l'algorithme de Dijkstra, pour obtenir la liste des poids des plus courts chemins issus du sommet A :

A	B	C	D	E
0	∞	∞	∞	∞

2. On considère le graphe orienté et valué positivement suivant :



- (a) Compléter le tableau d'exploration des sommets obtenu à l'aide de l'algorithme de Dijkstra, pour obtenir la liste des poids des plus courts chemins issus du sommet 3. Utiliser votre tableau pour donner un chemin de poids minimal reliant le sommet 3 au sommet 0.

0	1	2	3	4	5
∞	∞	∞	0	∞	∞

- (b) Reprendre la question précédente avec les chemins issus du sommet 0. Que constatez-vous ?

0	1	2	3	4	5
0	∞	∞	∞	∞	∞

Exercice 2 : Écriture de l'algorithme de Dijkstra

On pensera à importer le module `numpy` pour pouvoir définir des matrices en Python.

En cas de problème d'affichage des nombres réels dans la console, vous pouvez ajouter dans l'éditeur, la commande :

```
np.set_printoptions(legacy='1.25')
```

On cherche à écrire une fonction Python `Dijkstra(M,s)`, de paramètres `M` de type `array`, représentant la matrice des poids des arêtes (ou arcs) d'un graphe de sommets $\{0, 1, \dots, n-1\}$ et valué positivement, et `s` entier de $\llbracket 0, n-1 \rrbracket$ tel que tout sommet du graphe est accessible depuis `s`, telle que la fonction `Dijkstra` renvoie deux listes :

- l'une égale à $[\delta(s, 0), \delta(s, 1), \dots, \delta(s, n-1)]$, où l'on rappelle que $\delta(s, j)$ est le poids du chemin de poids minimal et reliant `s` à `j`,
- la seconde égale à $[i_0, i_1, \dots, i_{n-1}]$ où i_s est 'non', et pour $j \neq s$, i_j est le prédécesseur de `j` dans le chemin de poids minimal et reliant `s` à `j`, obtenu par l'algorithme.

Rappel :

Soit $G = (S, A, \nu)$ un graphe valué positivement, avec $S = \{0, 1, \dots, n-1\}$.

La matrice $M = (m_{i,j})_{0 \leq i,j \leq n-1}$ donnant les poids des arêtes (ou arcs) du graphe $G = (S, A, \nu)$ est définie par :

Si G est non orienté :

$$\forall (i, j) \in \llbracket 0, n-1 \rrbracket^2, m_{i,j} = \begin{cases} \nu(\{i, j\}) & \text{si } i \text{ et } j \text{ sont voisins} \\ \infty & \text{sinon} \end{cases}$$

Si G est orienté :

$$\forall (i, j) \in \llbracket 0, n-1 \rrbracket^2, m_{i,j} = \begin{cases} \nu((i, j)) & \text{si } j \text{ est un successeur de } i \\ \infty & \text{sinon} \end{cases}$$

1. En python, le symbole ∞ , traité comme $+\infty$, peut s'obtenir par la commande Python :

```
inf=float('inf')
```

Tester par exemple les commandes suivantes sur votre console :

```
inf=float('inf')
print(inf<10**6)
print(10**6<inf)
print(inf==inf+1)
print(inf<inf)
u=inf+1
print(u)
```

2. Écrire une fonction `initial` de paramètres `n` et `s` entiers tels que $0 \leq s < n$ et renvoyant trois listes :
- ★ l'une nommée `Satrait`, et égale à $[0, 1, \dots, s-1, s+1, \dots, n-1]$,
 - ★ la 2^{ième} nommée `Poids`, et égale à $[\text{inf}, \text{inf}, \dots, \text{inf}, 0, \text{inf}, \dots, \text{inf}]$, avec 0 en $s^{\text{ième}}$ position,
 - ★ la dernière nommée `Parent`, et égale à $[\text{inf}, \text{inf}, \dots, \text{inf}, \text{'non'}, \text{inf}, \dots, \text{inf}]$, avec 'non' en $s^{\text{ième}}$ position.
3. Écrire une fonction `voisin` de paramètres `M` de type `array`, représentant la matrice des poids des arêtes (ou arcs) d'un graphe de sommets $\{0, 1, \dots, n-1\}$ et valué positivement, `k` entier de $\llbracket 0, n-1 \rrbracket$, et `L` une liste d'entiers, et renvoyant la liste des voisins (ou successeurs) de `k`, distincts de `k` et qui appartiennent à la liste `L`, et renvoyant la liste vide si de tels voisins de `k` n'existent pas.

On complètera pour cela la fonction Python ci-dessous :

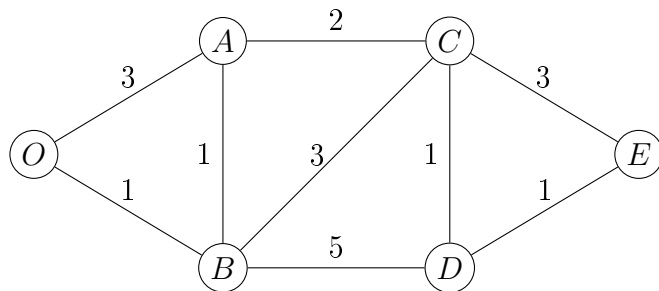
```
def voisin(M,k,L):
    u=[]
    n=len(M)
    for i in L:
        if **** and **** :
            ****
    return(u)
```

4. Écrire une fonction `echange` de paramètres `M` de type `array`, représentant la matrice des poids des arêtes (ou arcs) d'un graphe $G = (S, A, \nu)$ de sommets $\{0, 1, \dots, n-1\}$ et valué positivement, `k` et `j` deux entiers distincts de $\llbracket 0, n-1 \rrbracket$, `po` et `pa` deux listes de taille `n`, telle que :
- Si $po[j] > po[k] + \nu(k, j)$, alors :
 - on affecte à `po[j]` la valeur $po[k] + \nu(k, j)$,
 - et on affecte à `pa[j]` la valeur `k`.
 - Sinon, on ne fait rien
 - Renvoie les deux listes `po` et `pa` mises à jour.

5. Utiliser les fonctions précédentes pour écrire la fonction `Dijkstra`.
On pourra compléter la fonction Python ci-dessous :

```
def Dijkstra(M,s):
    n=len(M)
    Satraiter,Poids,Parent=initial(n,s)
    k=s
    while Satraiter!=[]:
        V= ****
        for j in V:
            exchange(M,k,j,Poids,Parent)
        r=0
        j=Satraiter[r]
        smin=j
        m=Poids[smin]
        while ****:
            r=r+1
            j=Satraiter[r]
            if ****:
                smin=j
                m=Poids[j]
            Satraiter.remove(smin)
        ****
    return(Poids,Parent)
```

6. (a) Définir la matrice M des poids des arêtes associée au graphe étudié en cours :

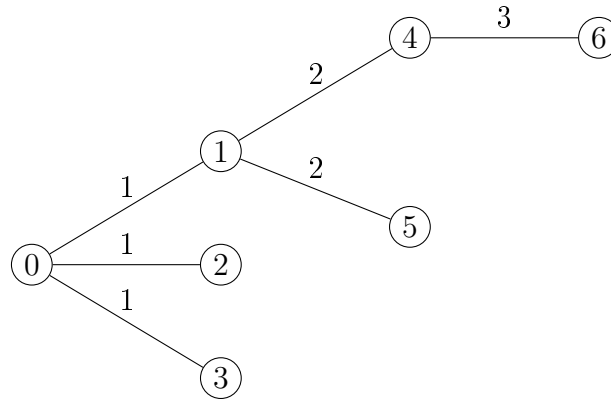


En python, on traduira le nom des sommets sous la forme :

$$\left\{ \begin{array}{ll} O & \rightarrow 0 \\ A & \rightarrow 1 \\ B & \rightarrow 2 \\ C & \rightarrow 3 \\ D & \rightarrow 4 \\ E & \rightarrow 5 \end{array} \right.$$

Puis tester votre fonction pour $s = O$.

(b) Définir la matrice M des poids des arêtes associée au graphe ci-dessous :

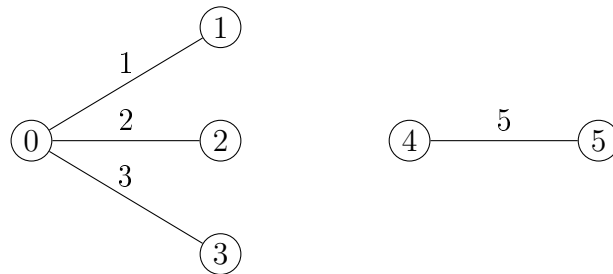


Puis tester votre fonction pour $s = 0$.

7. Lorsqu'un sommet t n'est pas accessible depuis s , on convient que $\delta(s, t) = +\infty$.

Expliquer pourquoi la fonction **Dijkstra** reste utilisable pour un graphe non (fortement) connexe et un sommet s qui n'est pas relié à chaque sommet du graphe.

On pourra tester la fonction **Dijkstra** avec le graphe orienté donné dans l'exercice 1, ou avec le graphe ci-dessous :



8. Utiliser la fonction **Dijkstra** afin d'écrire une fonction **chemin**, de paramètres M de type **array**, représentant la matrice des poids des arêtes (ou arcs) d'un graphe G de sommets $\{0, 1, \dots, n-1\}$ et valué positivement, et i et j deux sommets de G tels que j est accessible depuis i , et renvoyant la liste des sommets correspondant au chemin de poids minimal reliant i à j .

On pourra compléter la fonction Python ci-dessous :

```

def chemin(M, i, j):
    po, pa = Dijkstra(M, i)
    k = j
    u = [k]
    while *** :
        m = pa[k]
        ***
        ***
    return(u)

```

Tester votre fonction sur les graphes donnés dans l'exercice 1 et dans le présent exercice.

Exercice 3 : Une application de l'algorithme de Dijkstra

On donne la matrice ci-dessous qui donne les durées en heures de vols entre 6 villes, nommées $v_1, v_2, \dots, v_6$:

$$M = \begin{pmatrix} 0 & 3 & \infty & 5 & \infty & \infty \\ 3 & 0 & 5 & 2 & 4 & \infty \\ \infty & 4 & 0 & \infty & 4 & 3 \\ 6 & 2 & \infty & 0 & 4 & 4 \\ \infty & 4 & 4 & 5 & 0 & 2 \\ \infty & \infty & 5 & 4 & 3 & 0 \end{pmatrix}$$

c'est-à-dire que, pour tout $(i, j) \in \llbracket 1, 6 \rrbracket^2$, le coefficient $m_{i,j}$ de la matrice M situé à la $i^{\text{ième}}$ ligne et $j^{\text{ième}}$ colonne indique :

- la durée de vol en heures entre la ville v_i et la ville v_j , lorsque $m_{i,j} \in \mathbb{R}_+$,
 - que les villes v_i et v_j ne sont pas reliées par avion lorsque $m_{i,j} = \infty$.
1. En ne tenant pas compte des durées des escales, quel est l'itinéraire le plus rapide pour relier la ville v_1 à la ville v_6 ?
 2. On tient maintenant compte des durées d'escales qui sont données par :
$$\left\{ \begin{array}{ll} 2 \text{ h} & \text{si l'on change d'avion dans la ville } v_1 \\ 3 \text{ h} & \text{si l'on change d'avion dans la ville } v_2 \\ 1 \text{ h} & \text{si l'on change d'avion dans la ville } v_3 \\ 1 \text{ h} & \text{si l'on change d'avion dans la ville } v_4 \\ 4 \text{ h} & \text{si l'on change d'avion dans la ville } v_5 \\ 6 \text{ h} & \text{si l'on change d'avion dans la ville } v_6 \end{array} \right.$$

Quel est alors l'itinéraire le plus rapide pour relier la ville v_1 à la ville v_6 ?

Exercice 4 : L'algorithme de Maria Hasse

Soit $n \in \mathbb{N}^*$.

Lorsque $A = (a_{i,j})_{1 \leq i,j \leq n}$ et $B = (b_{i,j})_{1 \leq i,j \leq n}$ sont deux matrices de $\mathcal{M}_n(\mathbb{R} \cup \{\infty\})$, on définit la matrice $A \otimes B$ comme la matrice $C = (c_{i,j})_{1 \leq i,j \leq n}$ de $\mathcal{M}_n(\mathbb{R} \cup \{\infty\})$ telle que :

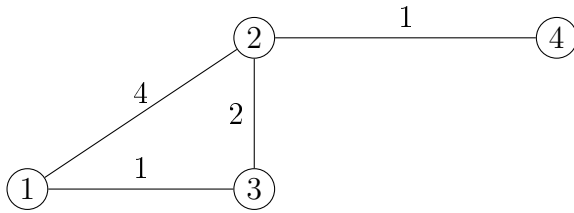
$$\forall (i,j) \in \llbracket 1, n \rrbracket^2, \quad c_{i,j} = \min_{1 \leq k \leq n} (a_{i,k} + b_{k,j})$$

Lorsque $A = (a_{i,j})_{1 \leq i,j \leq n} \in \mathcal{M}_n(\mathbb{R} \cup \{\infty\})$, on définit $A^{\otimes k}$, pour $k \in \mathbb{N}$, par :

$$A^{\otimes 0} = I_n, \quad A^{\otimes 1} = A, \quad \text{et si } k \geq 2, \quad A^{\otimes k} = A^{\otimes(k-1)} \otimes A$$

1. Étude d'un exemple

On considère le graphe valué $G = (S, A, \nu)$, où $S = \{1, 2, 3, 4\}$, donné ci-dessous :



- (a) Écrire la matrice $M = (m_{i,j})_{1 \leq i,j \leq 4}$ de $\mathcal{M}_4(\mathbb{R} \cup \{\infty\})$ définissant le poids des arêtes de G , c'est-à-dire telle que :

$$\forall (i,j) \in \llbracket 1, 4 \rrbracket^2, \quad m_{i,j} = \begin{cases} \nu(\{i,j\}) & \text{si } i \text{ et } j \text{ sont voisins} \\ 0 & \text{si } i = j \\ \infty & \text{sinon} \end{cases}$$

- (b) Calculer les matrices $M^{\otimes 2}$, $M^{\otimes 3}$ et $M^{\otimes 4}$.
(c) Pour chaque sommet s du graphe G , donner le poids du chemin de poids minimal reliant le sommet s à chacun des quatre sommets du graphes.
On pourra remplir les quatre tableaux ci-dessous :

Pour $s = 1$

1	2	3	4
0			

Pour $s = 2$

1	2	3	4
	0		

Pour $s = 3$

1	2	3	4
		0	

Pour $s = 4$

1	2	3	4
			0

Que constatez-vous ?

2. Soit $G = (S, A, \nu)$ un graphe d'ordre n , valué positivement, et ne possédant pas de boucle. On notera $1, 2, \dots, n$ les sommets de G .

On note M la matrice de $\mathcal{M}_n(\mathbb{R} \cup \{\infty\})$ donnant le poids des arêtes (ou arcs) comme dans l'exemple de la question précédente, une absence d'arête (ou d'arc) entre deux sommets distincts étant signalée par ∞ .

Les coefficients diagonaux de M sont tous égaux à 0.

- (a) Montrer que pour tout $k \in \mathbb{N}^*$, $M^{\otimes k}$ est la matrice de $\mathcal{M}_n(\mathbb{R} \cup \{\infty\})$ telle que, pour tout $(i, j) \in \llbracket 1, n \rrbracket^2$, le coefficient situé à la $i^{\text{ième}}$ ligne et $j^{\text{ième}}$ colonne est égal à :
- 0 si $i = j$,
 - ∞ si il n'existe pas de chemin (ou chaîne) de longueur $\leq k$ et reliant le sommet i au sommet $j \neq i$ de G ,
 - le poids du chemin de poids minimal et de longueur $\leq k$ et reliant le sommet i au sommet $j \neq i$ de G .
- (b) On note pour tout $k \in \mathbb{N}^*$, $m_{i,j}^{\otimes k}$ le coefficient de $M^{\otimes k}$ situé à la $i^{\text{ième}}$ ligne et $j^{\text{ième}}$ colonne. Justifier que : $\forall k \in \mathbb{N}^*, \forall (i, j) \in \llbracket 1, n \rrbracket^2, m_{i,j}^{\otimes(k+1)} \leq m_{i,j}^{\otimes k}$.
- (c) En déduire que si il existe $k \in \mathbb{N}^*$ tel que $M^{\otimes k} = M^{\otimes(k+1)}$, alors pour tout $(i, j) \in \llbracket 1, n \rrbracket^2$, $m_{i,j}^{\otimes k}$ désigne le poids d'un chemin de poids minimal et reliant le sommet i au sommet j .
- (d) Justifier qu'il existe $k \in \llbracket 1, n-1 \rrbracket$ tel que : $M^{\otimes k} = M^{\otimes(k+1)}$.

3. Application Python

- (a) Ecrire une fonction `Prod`, de paramètres `A` et `B` de type `array` et représentant deux matrices de $\mathcal{M}_n(\mathbb{R} \cup \infty)$, et renvoyant la matrice $A \otimes B$.

On rappelle que la commande Python

```
p , q = np . shape ( A )
```

appliquée à une matrice A , renvoie le couple (p, q) où : p est le nombre de lignes de A et q est le nombre de colonnes de A .

- (b) En déduire une fonction Python, nommée `Poids`, de paramètre `M` de type `array` qui représente la matrice associée à un graphe valué positivement (S, A, ν) où $S = \llbracket 0, 1, \dots, n-1 \rrbracket$, et `i, j` représentant deux entiers de S , et qui renvoie la valeur de $\delta(i, j)$, le poids du plus court chemin reliant i et j si un tel chemin existe, et `inf` sinon.

On pourra utiliser le test en Python, pour deux matrices de même taille `A` et `B`

```
np . allclose ( A , B ) == False
```

qui renvoie $\begin{cases} \text{True} & \text{lorsque les matrices } A \text{ et } B \text{ ne sont pas égales} \\ \text{False} & \text{lorsque les matrices } A \text{ et } B \text{ sont égales} \end{cases}$
